Procedural approach	OOP approach
① line by line execution.	① organized into classes & objects
② we can use only built in data types list, tuple, set, dict int, float, bool, complex, str	② we can create our own custom data type.
③ used for small scripts.	③ used for large applications.
④ 1000 lines of code	⑤ Half the code - 500 lines of code

- Everything in python is an object

eg. nums = [10, 20, 30]
nums.append(40)

list, tuple, set, dict - class

int, float, complex, bool, str - class

- create object of class

Syntax :-

obj_name = class_name()

list1 = list() ← empty list

OOP has 6 Principles :-

① class & object -

② Encapsulation -
③ Polymorphism -
④ Inheritance -
⑤ Abstraction -
⑥ Magic method -

iterator, Generator, Decorator -

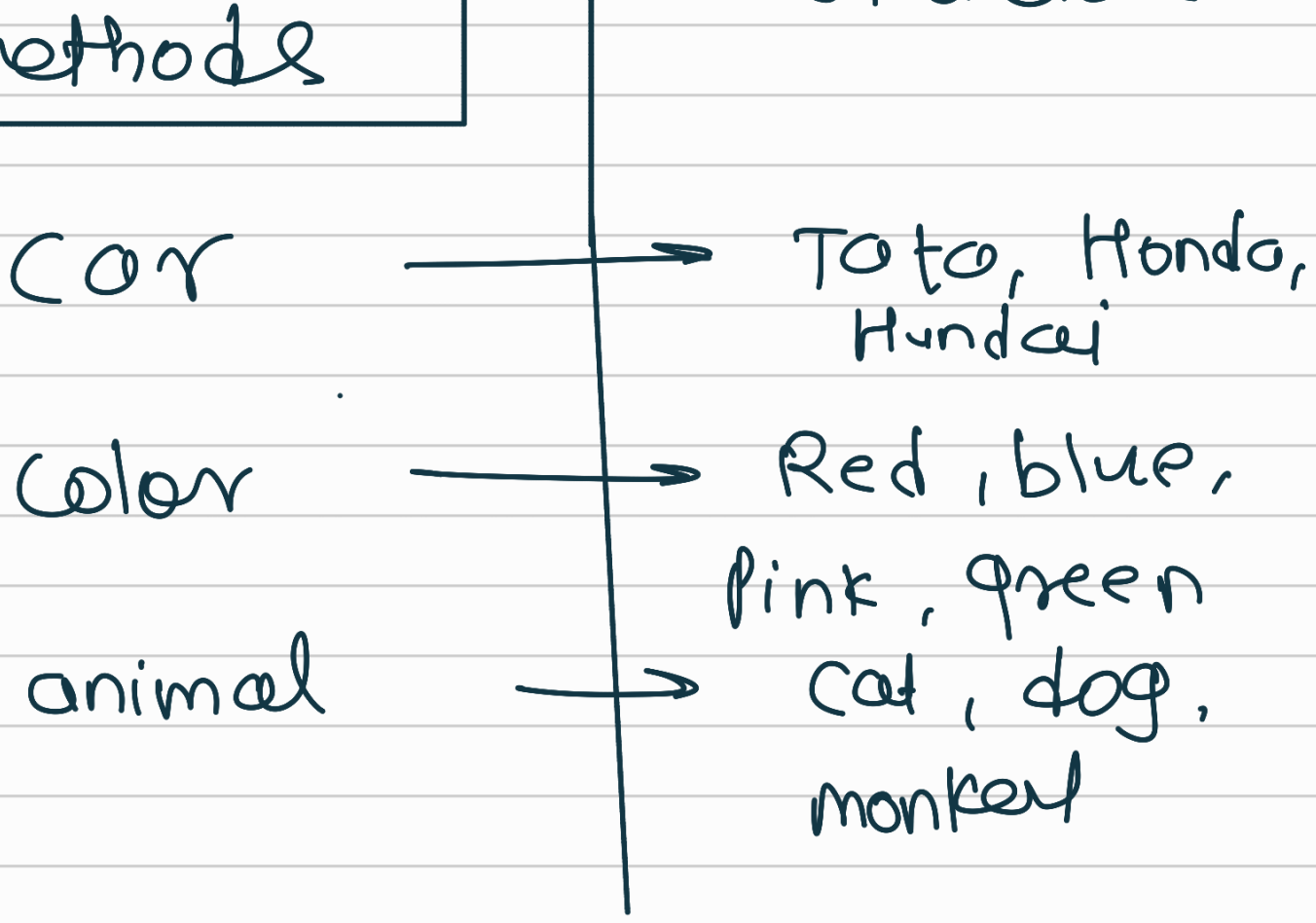
1] class & object -

class - class is a set of rules that define how its object will behave. - Blueprint

object - object is an entity that follows class rules.
- Instance of class

class (Blueprint)	object Instance
Attributes +	1) follow rules of a class

methods



Syntax :

```
class class_name:
    def __init__():
    def fun_name():
        ==========
```

class

```
obj_name = class_name()
obj_name.fun_name()
```

— object of class

class - Two Components

1) Attribute | 2) Methods

Data/properties

behaviour/function

① Car class –

color, milage,
Speed, price

② avg_speed()
milage()

② ATM class –

Pincode,
account balance

withdrawal() ↓
deposite() ↑

③ Student class

name, roll no,
address,
cgpa

display_attendance()

display_report_card()

eg.

```
class Vehical:
    def __init__(self):
        self.color = "red"
        self.top-speed = 150
        self.mileage = 15
```

attribute, data properties

```
    def display_info(self):
        print("milage is",
              self.mibage)
        print("top speed is",
              self.top-speed)
```

method self fun

```
tata = Vehical()
print(tata.color)  → Red
print(tata.top-speed) - 150
print(tata.mileage) - 15
```

tata.display_info() ⇒
milage is - 15
top speed is 150

29.Python_Class_and_objects_13oct > vehical_multiple_objects.py > ...

```python
class Vehical:
    def __init__(self,color,top_speed,mileage):# constructor
        self.color=color,
        self.top_speed=top_speed
        self.mileage=mileage
    def display_info(self):
        print("color of car is",self.color)
        print("top speed of car is",self.top_speed)
        print("mileage of car is",self.mileage)

tata=Vehical("white",150,20)
honda=Vehical("grey",180,15)
mahindra=Vehical("black",160,17)
```

obj_name = class_name(_, _, _)

vehical

	tata	honda	mahinda
Colour →	white	grey	black
topspeed →	150	180	160
mileage →	20	15	17

a = 10

print(a)